



Voraxia Camera System Development Plan

Pivot-based, runtime-blended, deterministic third-person camera architecture

Version: Current

Date: 29 June 2026

Prepared By: Coding Custard Studios

Project: Voraxia

Status: Development Plan

Coding Custard Studios | Voraxia Documentation Standard

1. Purpose and Source Reference

This document converts the PloxTools manual and our Voraxia camera requirements into a full, staged development plan. It is designed to guide implementation before code starts, so we avoid turning the camera into a single haunted .cpp crypt.

Source material used: the uploaded **Framing.pdf** manual, including its sections on framing, rotation, collision, focus, spline cameras, movement anticipation, pitch/speed effects, camera animations, camera shake, control/view handling, dithering, settings assets, and debug tools.

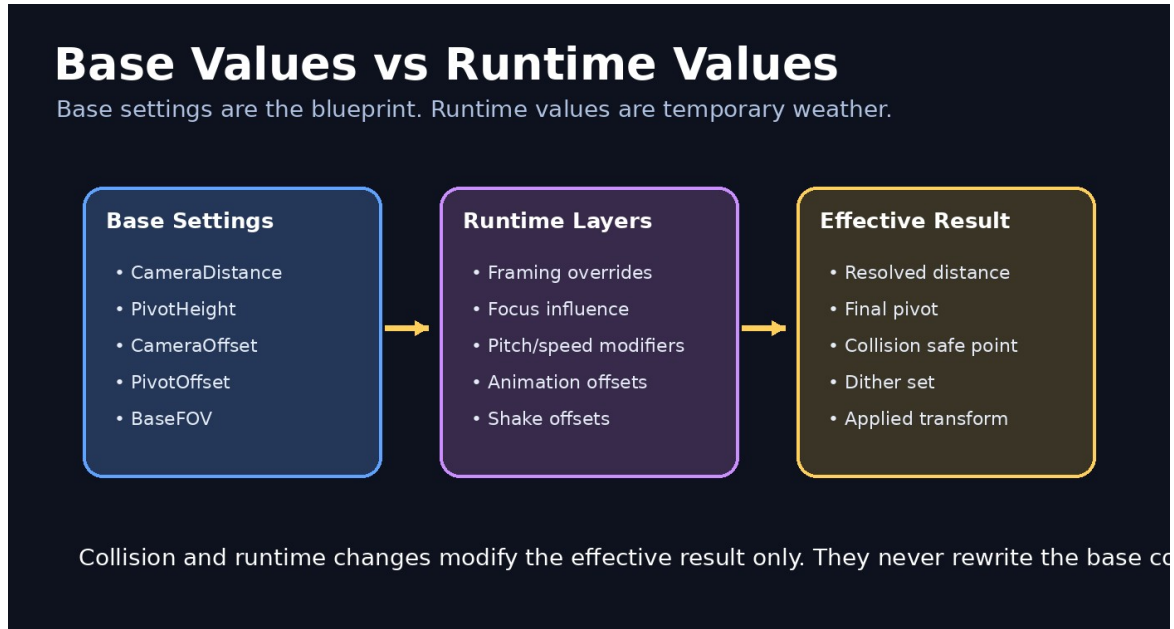
Voraxia-specific rules layered on top:

- **No SpringArm.** The camera transform is calculated manually each frame.
- The camera is built around a **pivot point**, which defines the origin for orbit, distance, rotation and target framing.
- Base values are never mutated by gameplay events. Runtime values are temporary, blendable and resettable.
- Focus and spline are camera states, not helper functions.
- Collision and dithering are final safety/visibility passes, not replacements for framing.
- Blueprint and C++ control must be available for all production-facing systems.

Design motto

The camera is calculated, not attached.
Every system contributes to a final solved transform.

2. Core Camera Model



At the heart of the system is a simple but strict model: the actor provides the anchor, the camera component computes a pivot, rotation determines the orbit direction, framing determines the desired camera position, modifiers add temporary gameplay feel, and collision/occlusion produce the safe final result.

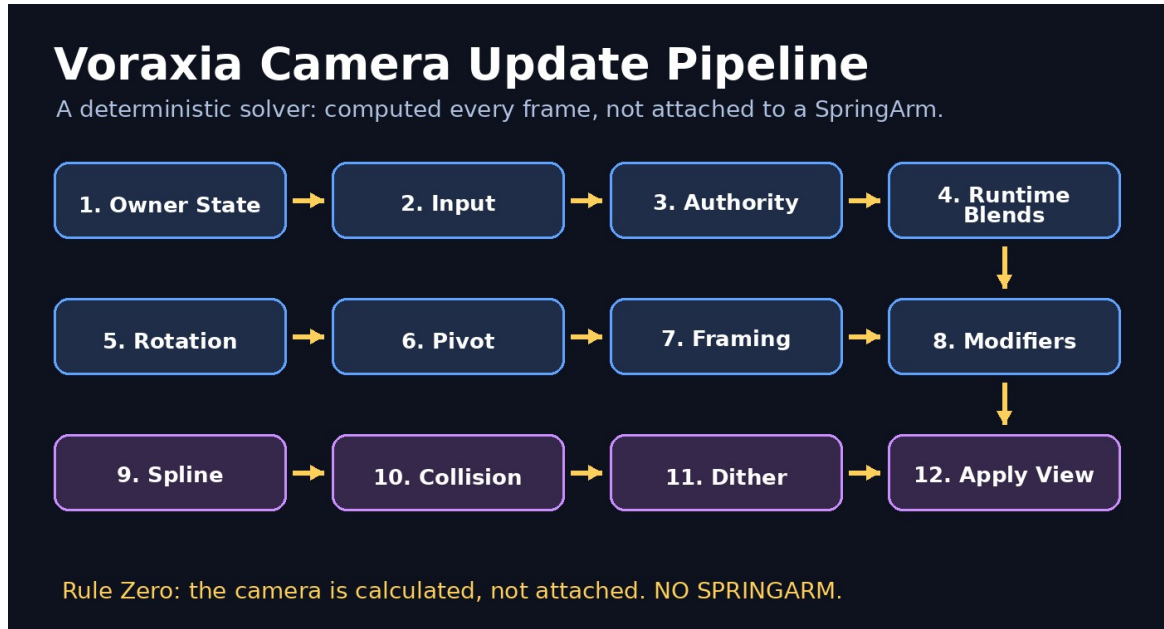
Core transform concept

```

Pivot = OwnerLocation + PivotHeight + PivotOffset
DesiredCamera = Pivot - CameraForward * EffectiveDistance + CameraOffset
FinalCamera = CollisionAndOcclusionResolved(DesiredCamera)
FinalRotation = SolvedRotation
  
```

Camera distance is the unobstructed desired distance. Collision can compress the camera closer to the pivot, but it does not replace or edit the base distance.

3. Master Update Pipeline



The update order is fixed so features remain predictable. This is especially important once focus, spline mode, collision feelers, additive animations and dither restoration all exist at once.

Stage	Responsibility
1. Gather owner state	Location, velocity, acceleration/input direction, movement mode and character movement state.
2. Gather input	Yaw/pitch input, look timers, input enabled flags.
3. Resolve authority	Disabled, external view override, spline, focus or gameplay camera.
4. Update blends	Distance, pivot height, offsets, FOV, constraints, focus and spline transitions.
5. Solve rotation	Input, yaw follow, pitch follow, vertical support, ceiling anticipation, focus influence, constraints and lag.
6. Solve pivot	Owner location plus base/runtime pivot height and pivot offset.
7. Solve framing	Camera distance, camera offset, FOV, shoulder state and base/runtime values.
8. Apply modifiers	Movement anticipation, pitch effects, speed effects, camera animations and shakes.
9. Resolve spline	Timed or tethered spline mode can override gameplay transform.
10. Resolve collision	Main sweep, feelers, compression, smoothing, recovery and ignored actors/tags.
11. Resolve dithering	Detect blockers, fade targets, restore previous blockers.
12. Apply view	Set world location/rotation/FOV or route view camera blending.

4. Authority and Priority Rules

Camera Authority Priority

Higher layers override lower layers. Additive layers still stack after gameplay solving.

- 1 **Disabled / External View Override**
The camera component stops or hands the rendered view to another camera.

- 2 **Spline Mode**
Scripted paths temporarily replace gameplay camera positioning.

- 3 **Focus Mode**
Target tracking alters rotation behavior: hard lock or soft lock.

- 4 **Gameplay Camera**
Normal pivot, rotation, framing, follow, and modifiers.

- 5 **Animation + Shake**
Additive authored motion and procedural feedback.

- 6 **Collision + Dithering**
Final safety pass protects visibility and avoids clipping.

Priority rules prevent feature soup. Spline mode wins over focus. Focus wins over gameplay follow. Additive animations and shakes stack late. Collision and dithering happen at the end because they are safety and readability systems.

Rule of thumb

If a system describes where the camera should be, it belongs before collision. If a system protects visibility or prevents clipping, it belongs after the desired camera transform has been solved.

5. Development Roadmap



We should build this in slices, not as one roaring megafire. The first slice must make a playable camera with clean base/runtime framing. Only then do we feed it collision, focus, spline rails and cinematic thunder.

Milestone	Main outcome	Systems included
0	Plugin shell and spec	Module, logging, component shell, class naming, folder layout.
1	Core camera hatchling	Pivot, distance, offsets, FOV offset, shoulder swap, target camera reference.
2	Rotation brain	Input, lag, yaw/pitch follow, vertical support, ceiling anticipation, constraints.
3	Collision claws	Pivot sweep, compression, recovery, predictive feelers, ignored actors/tags.
4	Focus eyes	Actor/component/socket/location tracking, hard/soft lock, auto-clear, events.
5	Spline rails	Timed and tethered camera splines, blend in/out, return to gameplay camera.
6	Dynamic framing muscles	Movement anticipation, pitch-based effects, speed-based effects.
7	Occlusion smoke	Dither detection, owner/attached fading, tagged blockers, material support.
8	Juice layer	Camera animations, camera shakes, handles, callbacks.
9	Production polish	View camera, settings assets, debug tools and runtime control utilities.

6. Framing Core

Framing



Framing reference, p. 1

Voraxia interpretation

Framing defines the camera's default spatial relationship to the character. Voraxia keeps the same conceptual split: camera distance describes the unobstructed target distance, pivot height/pivot offset define the orbit origin, and camera offset changes final composition without moving the pivot.

Settings / State

- CameraDistance, PivotHeight, CameraOffset, PivotOffset, BaseFOV.
- Runtime versions of distance, pivot height, camera offset, pivot offset and FOV offset.
- BlendTime and optional CurveFloat for every runtime transition.
- Shoulder switching by flipping runtime camera-offset Y, not by editing base values.
- Camera offset is clamped inside allowed distance; increasing distance is how we move farther away.

Public API

- SetFraming / ResetFraming.
- SetCameraDistance / ResetCameraDistance.
- SetPivotHeight / ResetPivotHeight.
- SetCameraOffset / ResetCameraOffset.
- SetPivotOffset / ResetPivotOffset.
- SetFOVOffset / ResetFOVOffset.
- SwapCameraShoulder.

Definition of done

- Camera orbits around a calculated pivot.
- Pivot offset changes orbit center; camera offset changes composition.
- FOV offset changes current FOV without altering base FOV.
- Runtime reset restores original component settings.

7. Rotation and Constraints

Rotation



Rotation reference, p. 11

Voraxia interpretation

Rotation is a layered solver: player look input first, then movement-aware follow systems, vertical/collision-aware pitch helpers, constraints and finally lag. Follow systems should only activate after look input has stopped long enough to avoid camera nagging.

Settings / State

- RotationLagSpeed and bEnableRotationLag.
- RestingCameraPitch, pitch follow speed, pitch follow delay/angle/speed thresholds.
- Yaw follow speed, diagonal strength, delay/angle/speed thresholds.
- Vertical pitch support for slopes and falling.
- Ceiling anticipation traces above/forward from pivot.
- Pitch/yaw constraints with soft tolerance zones and blend time.

Definition of done

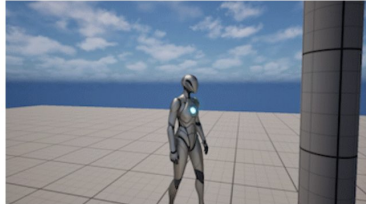
- Manual rotation is responsive.
- Yaw/pitch recenter only after input silence.
- Constraints slow near limits instead of hard snapping.
- Ceiling anticipation can bias pitch downward in tight spaces.

Public API

- AddYawInput / AddPitchInput.
- SetCameraRotation.
- ResetCameraRotation.
- SetPitchConstraints.
- SetYawConstraints.
- Enable/disable look input.
- SetCharacterRotationFollowsCamera.

8. Collision and Predictive Avoidance

Collision



Collision reference, p. 24

Voraxia interpretation

Collision runs after framing and modifiers. The camera traces from pivot to desired camera position, computes a safe distance, compresses toward that distance and recovers smoothly when clear. Predictive feelers probe nearby directions before the main camera path becomes blocked.

Settings / State

- `bEnableCameraCollision`.
- `CameraCollisionRadius` and `MinDistanceFromPivot`.
- `CollisionCompressionSpeed`, `CollisionDistanceSmoothingSpeed`, `CollisionRecoverySpeed`.
- `IgnoreCharacters` and `IgnoreCollisionTags`.
- `bEnablePredictiveAvoidance`, `FeelerYawOffset`, `FeelerPairs`, `FeelerProbeRadius`.
- Debug for main sweep and feeler sweeps.

Definition of done

- Camera compresses closer to pivot when blocked.
- Camera recovers smoothly when clear.
- Predictive feelers reduce sudden clipping during fast turns.
- Base `CameraDistance` remains unchanged.

Public API

- `SetCameraCollisionEnabled`.
- `SetCollisionIgnoredTags`.
- `AddIgnoredActor` / `RemoveIgnoredActor`.
- `SetPredictiveAvoidanceEnabled`.
- `GetDesiredCameraDistance`.
- `GetEffectiveCameraDistance`.
- `GetLastCollisionHit`.

9. Focus System

FOCUS



Focus reference, p. 31

Voraxia interpretation

Focus is a temporary camera state. While active, it alters rotation behavior and tracks an Actor, Component, Socket or Location. Hard lock centers the target with strict rotation; soft lock allows controlled player deviation before returning.

Settings / State

- FocusMode: HardLock or SoftLock.
- Auto clear by distance and visibility, with visibility delay.
- HardLockRotationSpeed.
- SoftYawLimit, SoftPitchLimit and SoftReturnSpeed.
- Optional hard yaw/pitch limits inside soft lock.
- Current focus target type, target data, validity and distance from pivot.

Public API

- SetFocusActor.
- SetFocusActorSocket.
- SetFocusActorComponent.
- SetFocusLocation.
- ClearFocus.
- IsFocusActive / GetCurrentFocus.
- OnFocusStarted, OnFocusChanged, OnFocusFinished.

Definition of done

- Hard lock rotates toward target predictably.
- Soft lock blends player input with target tracking.
- Focus can auto-clear by distance, visibility or invalid target.
- Focus events expose previous/new focus and end reason.

10. Spline Camera

Spline



Spline reference, p. 44

Voraxia interpretation

Spline camera is a temporary mode that can override gameplay camera behavior. Timed mode travels along a spline over duration. Tethered mode constrains camera movement to the spline while following player progress.

Settings / State

- AVoraxiaCameraSplineActor using USplineComponent.
- SplineMode: Timed or Tethered.
- Duration, BlendInTime, BlendOutTime and bReturnToGameplayCamera.
- Rotation modes: spline rotation, look at player, look at focus, look at location, blend gameplay rotation.
- Runtime overrides for individual play requests.
- Spline alpha/progress state.

Public API

- PlayCameraSpline.
- StopCameraSpline.
- IsCameraSplineActive.
- GetActiveCameraSpline.
- SetSplineRotationTarget.
- OnSplineFinished.

Definition of done

- Timed spline plays start to end.
- Tethered spline follows player position/progress.
- Spline cleanly overrides gameplay camera.
- Return to gameplay camera blends smoothly.

11. Movement Anticipation and Dynamic Modifiers

Movement Anticipation



Movement anticipation reference, p. 58

Movement anticipation

Movement anticipation shifts the camera toward movement direction to improve forward readability and enhance speed feel. It is fully dynamic, activates after look input has paused, scales by speed/input and returns smoothly to zero when movement stops.

- AnticipationStrength and interpolation speed.
- LookInputDelay.
- AnticipationFullSpeed.
- Optional lateral-only mode.
- Never modifies base framing.

Pitch Based Effects



The **Pitch Based Effects** system dynamically adjusts camera distance and

Pitch effects reference, p. 61

Speed Based Effects



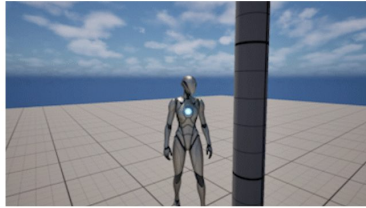
The **Speed Based Effects** system adjusts camera distance and field of view

Speed effects reference, p. 64

Modifier	Voraxia behavior
Pitch distance	Adjust desired distance when looking up/down.
Pitch FOV	Apply FOV percentage/offset based on pitch.
Speed distance	Shift camera distance based on movement speed between min and max thresholds.
Speed FOV	Increase/decrease FOV with movement speed, optionally shaped by a curve.

12. Dithering and Occlusion

Dithering



Object
Dithering reference, p. 100

Voraxia interpretation

Dithering protects visibility without yanking the camera. Tagged blockers between pivot and camera can fade out, the owner can fade when the camera gets very close, and attached actors such as weapons can fade with the character.

- Requires material support or a Voraxia dither interface.
- Actors/components are grouped so multi-component objects fade as one visual object.
- Objects restore automatically when no longer blocking.
- Manual reset exists for safety after streaming/destruction edge cases.

Setting	Purpose
EnableDithering	Turns the system on/off.
DitherOwner	Allows the player character to fade at close range.
DitherAttachedActors	Allows weapons/equipment to fade with the owner.
DitherTags	Only matching tagged actors are affected.
DitherMinOpacity	Lowest opacity during fade.
DitherSpeed	Fade in/out speed.
DitherSweepRadius	Trace radius for blockers between pivot and camera.
DitherOverlapRadius	Close-range overlap radius around camera.

13. Camera Animations and Shake

Camera Animations

Camera Shake



The **Camera Animations** system allows you to play procedural camera
Camera animation reference, p. 68



The **Camera Shake** system adds procedural camera motion for gameplay
Camera shake reference, p. 76

Camera animations and shakes are additive layers, not camera modes. They should never overwrite focus, spline, collision or base framing. Use animations for authored motion and shake for procedural feedback.

System	Behavior	API
Camera Animation	Relative transform motion applied on top of gameplay camera; multiple animations can play simultaneously; each returns a handle.	PlayCameraAnimation, StopCameraAnimation, StopAllCameraAnimations, IsCameraAnimationPlaying.
Camera Shake	Uses Unreal camera shake patterns for idle, movement, impacts, recoil and environmental effects; can be automatic or manual.	PlayCameraShake, StopCameraShake, StopAllCameraShakes.

Automatic shake candidates

- Idle shake: subtle low-intensity motion when stationary.
- Movement shake: speed-scaled motion using min/max speed, min/max scale, interpolation speed and optional curve.
- Manual shake: impacts, explosions, weapon recoil, scripted events.

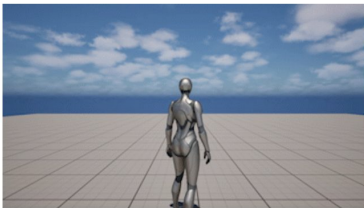
14. Camera Control, View, Settings Assets and Debug

Camera Control



The **Camera Control** section provides runtime functions for controlling the
Camera control reference, p. 85

Camera View



The **Camera View** system is responsible for switching the camera used as the
Camera view reference, p. 94

Control utilities temporarily adjust the internal camera behavior. View functions control what the player sees. These must remain separate: changing a target camera should not automatically change the rendered view camera, and changing the view should not corrupt internal camera state.

Settings Asset

The **Settings Asset** system allows you to store and reuse complete camera configurations.

A Settings Asset is a data asset that contains all camera parameters in a single place.

It can be used both:

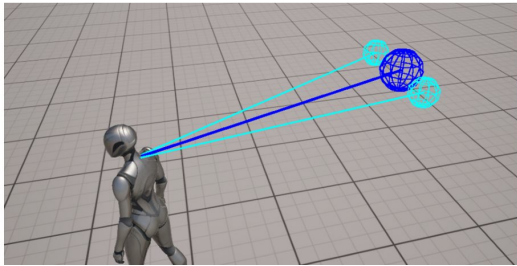
- at runtime to switch camera behavior dynamically
- in the editor for fast setup, iteration, and preset selection

This makes it easy to:

- switch between different gameplay states (combat, exploration, sprinting)

Settings asset reference, p. 110

Debug



Debug reference, p. 117

Area	Voraxia deliverables
Control	SetTargetCamera, SetCameraRotation, ResetCameraRotation, enable/disable system/look/move/player control, character follows camera yaw.
View	SetViewCamera, BlendToViewCamera, GetCurrentViewCamera, GetViewTargetCamera.
Settings Asset	Reusable data asset for complete camera configurations; instant apply plus smooth framing apply.
Debug	Draw pivot, ideal camera, final camera, sweeps, feelers, focus target, spline sample and dither blockers.

15. Feature Map

System	Voraxia behavior	Build
Framing	Distance, pivot height, camera/pivot offsets, FOV offset, shoulder swap.	Milestone 1
Runtime blending	Blend time plus optional curve across values and states.	Milestone 1
Rotation lag	Interpolated rotation toward target rotation.	Milestone 2
Yaw/Pitch follow	Recenters when player stops look input and movement thresholds are met.	Milestone 2
Vertical support	Pitch offsets from slopes/falling using CharacterMovement when available.	Milestone 2
Ceiling anticipation	Traces upward/forward and biases pitch downward.	Milestone 2
Constraints	Soft pitch/yaw zones with blendable limits.	Milestone 2
Collision	Pivot-to-camera sweep, compression, smoothing, recovery.	Milestone 3
Predictive feelers	Side probes around desired camera direction.	Milestone 3
Focus	Actor, component, socket and location targets; hard/soft lock.	Milestone 4
Spline	Timed and tethered spline override modes.	Milestone 5
Movement anticipation	Movement-direction framing offset.	Milestone 6
Pitch/speed effects	Runtime-only distance/FOV modifiers.	Milestone 6
Dithering	Fades blockers, owner and attached actors.	Milestone 7
Animations/shake	Additive authored/procedural motion.	Milestone 8
Control/view	Target camera, view camera, input control, settings assets, debug.	Milestone 9

16. Public API Surface

Category	Blueprint/C++ functions
Framing	SetFraming ResetFraming SetCameraDistance SetPivotHeight SetCameraOffset SetPivotOffset SetFOVOffset SwapCameraShoulder
Rotation	AddYawInput AddPitchInput SetCameraRotation ResetCameraRotation SetPitchConstraints SetYawConstraints
Collision	SetCameraCollisionEnabled SetPredictiveAvoidanceEnabled AddIgnoredActor RemoveIgnoredActor SetIgnoredTags
Focus	SetFocusActor SetFocusActorSocket SetFocusActorComponent SetFocusLocation ClearFocus GetCurrentFocus
Spline	PlayCameraSpline StopCameraSpline IsCameraSplineActive GetActiveCameraSpline
Animation/Shake	PlayCameraAnimation StopCameraAnimation StopAllCameraAnimations PlayCameraShake StopCameraShake StopAllCameraShakes
Control/View	SetTargetCamera SetViewCamera BlendToViewCamera SetCameraSystemEnabled SetLookInputEnabled SetPlayerControlEnabled
Settings/Debug	ApplyCameraSettingsAsset ApplyCameraSettingsAssetSmooth SetDebugEnabled SetDebugDrawFlags

Naming note

The API should feel familiar to the PloxTools behavior reference, but should use Voraxia names, Unreal style and our own implementation internals.

17. Suggested C++ File Layout

```

Plugins/VoraxiaCamera/
Source/VoraxiaCameraRuntime/
Public/VoraxiaCameraComponent.h
Public/VoraxiaCameraSplineActor.h
Public/VoraxiaCameraSettingsAsset.h
Public/VoraxiaCameraTypes.h
Private/VoraxiaCameraComponent.cpp
Private/VoraxiaCameraSplineActor.cpp
Private/VoraxiaCameraSettingsAsset.cpp
Private/VoraxiaCameraDebug.cpp

```

Recommended type groups

Type group	Examples
Settings structs	FVoraxiaCameraFramingSettings, RotationSettings, CollisionSettings, FocusSettings, SplineSettings, DitheringSettings.
Runtime state structs	FramingState, RotationState, CollisionState, FocusState, SplineState, DitherState, AnimationState.
Blend helpers	FVoraxiaRuntimeFloat, FVoraxiaRuntimeVector, FVoraxiaRuntimeRotator.
Events/data	FVoraxiaCameraFocusData, EFocusEndReason, FVoraxiaCameraSplinePlayParams.
Debug	EVoraxiaCameraDebugFlags and debug drawing helpers.

Module API macro

If this is a separate plugin module, use the plugin module macro generated by Unreal. If we place it in the main Voraxia module, use the existing Voraxia module API macro.

18. First Implementation Slice

This is the first code slice I recommend when we begin. It deliberately avoids collision, focus and spline so we can verify the mathematical spine first.

Piece	Details
Component	UVoraxiaCameraComponent with tick, target camera reference and no SpringArm dependency.
Runtime blends	Float, vector and FOV runtime values with base/current/target/start/elapsed/curve.
Framing settings	CameraDistance, PivotHeight, CameraOffset, PivotOffset, BaseFOV.
Rotation settings	Yaw/pitch input speeds, min/max pitch, optional rotation lag.
Transform solver	Calculate pivot, desired camera position, final camera transform and final FOV.
Blueprint API	Set/reset framing values, shoulder swap, AddYawInput/AddPitchInput, SetTargetCamera.
Debug basics	Draw pivot, desired camera and final camera.

Acceptance test

- Character uses a CameraComponent directly attached to root or mesh, but the component sets its world transform manually.
- Changing CameraDistance moves the camera closer/farther from the pivot.
- Changing PivotOffset moves the orbit center.
- Changing CameraOffset changes composition without changing orbit center.
- ResetFraming restores base values.
- No SpringArm exists in the setup.

19. Risks, Decisions and Guardrails

Risk	Guardrail
Megafire creep	Keep settings/state/types in separate headers and implementation helpers grouped by system.
Focus fighting spline	Authority priority: spline wins over focus.
Runtime values corrupting base settings	Base/current/target model, plus explicit reset functions and tests.
Collision jitter	Separate desired distance, safe distance and smoothed effective distance.
Dither material complexity	Start with detection/debug first, then dynamic material instances, then optional interface.
Unreal Character dependency	Support generic Actor owners, but use CharacterMovement data only when available.
Hard-to-debug camera feel	Build debug drawing from milestone 1, not later.
View/target confusion	Target camera is what we control; view camera is what the player sees.

Open decisions

- Plugin vs main module: recommended plugin for separation.
- Component name: UVoraxiaCameraComponent is preferred because the full system is larger than pivot framing.
- Settings asset format: DataAsset with all system settings, plus smooth framing-only application.
- Dithering approach: dynamic material instances first, interface later if needed.
- Spline tether method: closest point to player vs player-progress projection. We can prototype both.

20. One-Page Build Checklist

Phase	Checklist
Phase 0	Create plugin/module, log category, component shell, folder structure.
Phase 1	Implement framing, runtime blends, FOV offset, shoulder swap, debug pivot/desire/final.
Phase 2	Add rotation input, lag, constraints, yaw/pitch follow, vertical pitch and ceiling anticipation.
Phase 3	Add collision sweep, compression/recovery, ignored actors/tags and predictive feelers.
Phase 4	Add focus state, target types, hard/soft lock, auto-clear and events.
Phase 5	Add spline actor, timed/tethered playback, blend in/out and rotation modes.
Phase 6	Add movement anticipation, pitch-based modifiers and speed-based modifiers.
Phase 7	Add dither detection, material integration, owner/attached fading and restore logic.
Phase 8	Add camera animation and camera shake wrappers with handles/callbacks.
Phase 9	Add view camera helpers, settings assets, full debug and tuning polish.

Final forge note: build the camera in playable slices. A camera system only becomes trustworthy when each layer is visible, testable and reversible.